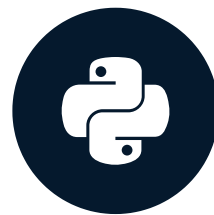


Portfolio composition and backtesting

INTRODUCTION TO PORTFOLIO RISK MANAGEMENT IN PYTHON



Dakota Wixom

Quantitative Analyst | QuantCourse.com

Calculating portfolio returns

Portfolio Return Formula:

$$R_p = R_{a_1}w_{a_1} + R_{a_2}w_{a_2} + \dots + R_{a_n}w_{a_n}$$

- R_p : Portfolio return
- R_{a_n} : Return for asset n
- w_{a_n} : Weight for asset n

Assuming `StockReturns` is a `pandas` `DataFrame` of stock returns, you can calculate the portfolio return for a set of portfolio weights as follows:

```
import numpy as np
portfolio_weights = np.array([0.25, 0.35, 0.10, 0.20, 0.10])
port_ret = StockReturns.mul(portfolio_weights, axis=1).sum(axis=1)
port_ret
```

```
Date
2017-01-03    0.008082
2017-01-04    0.000161
2017-01-05    0.003448
...
```

```
StockReturns["Portfolio"] = port_ret
```

Equally weighted portfolios in Python

Assuming `StockReturns` is a `pandas` `DataFrame` of stock returns, you can calculate the portfolio return for an **equally weighted** portfolio as follows:

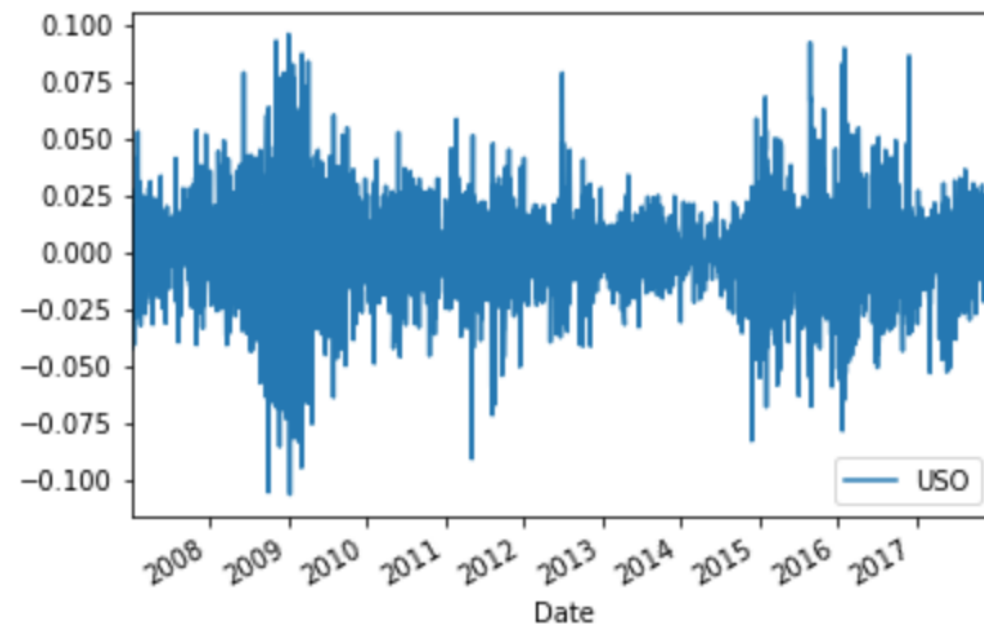
```
import numpy as np
numstocks = 5
portfolio_weights_ew = np.repeat(1/numstocks, numstocks)
StockReturns.iloc[:,0:numstocks].mul(portfolio_weights_ew, axis=1).sum(axis=1)
```

```
Date
2017-01-03    0.008082
2017-01-04    0.000161
2017-01-05    0.003448
...
```

Plotting portfolio returns in Python

To plot the daily returns in Python:

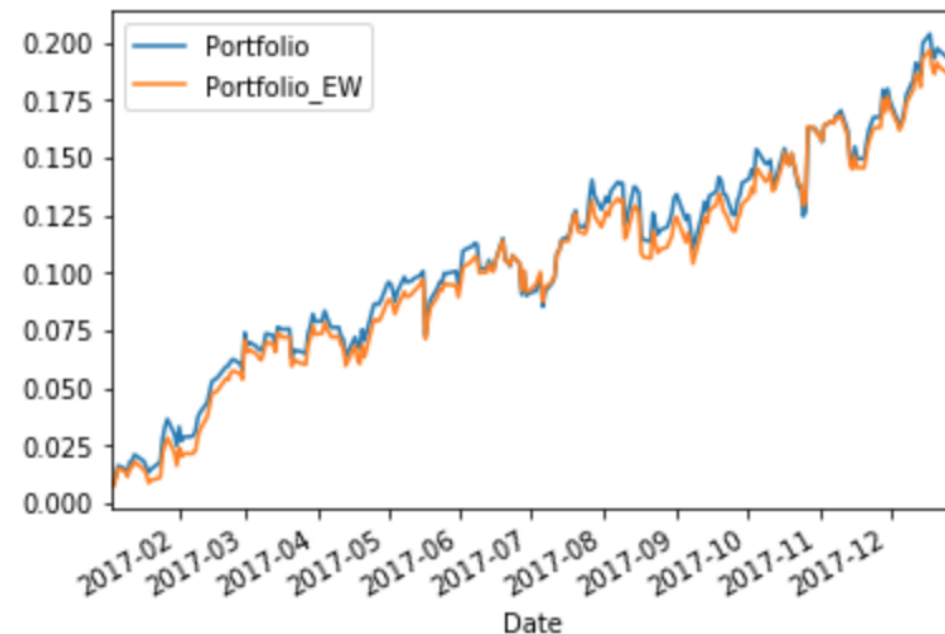
```
StockPrices["Returns"] = StockPrices["Adj Close"].pct_change()  
StockReturns = StockPrices["Returns"]  
StockReturns.plot()
```



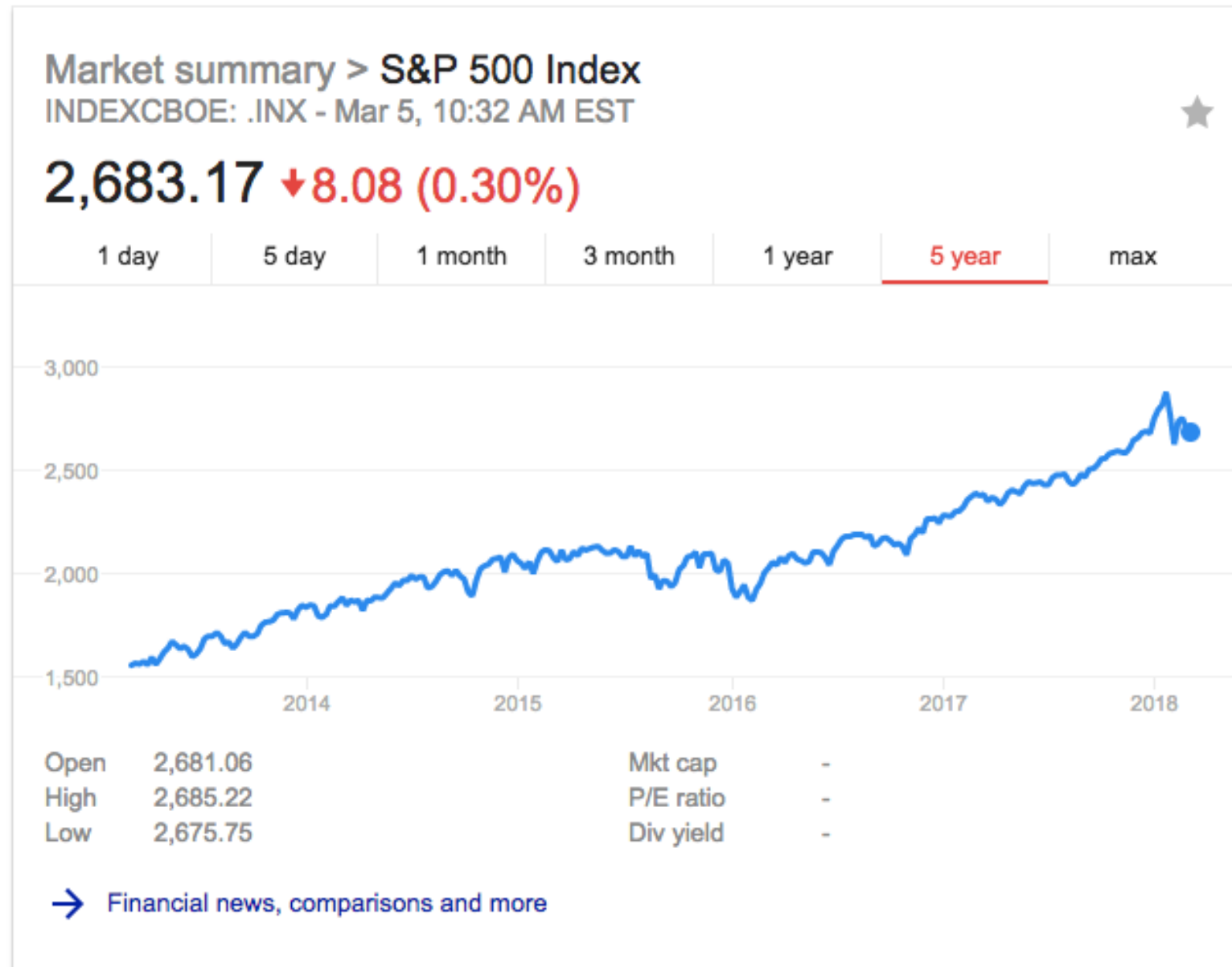
Plotting portfolio cumulative returns

In order to plot the cumulative returns of multiple portfolios:

```
import matplotlib.pyplot as plt  
CumulativeReturns = ((1 + StockReturns).cumprod() - 1)  
CumulativeReturns[["Portfolio", "Portfolio_EW"]].plot()
```



Market capitalization



Market capitalization

Market capitalization: The value of a company's publicly traded shares.

Also referred to as **Market cap**.

Market-cap weighted portfolios

In order to calculate the market cap weight of a given stock n :

$$w_{mcap_n} = \frac{mcap_n}{\sum_{i=1}^n mcap_i}$$

Market-Cap weights in Python

To calculate market cap weights in python, assuming you have data on the market caps of each company:

```
import numpy as np
market_capitalizations = np.array([100, 200, 100, 100])
mcap_weights = market_capitalizations/sum(market_capitalizations)
mcap_weights
```

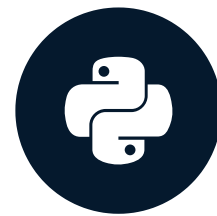
```
array([0.2, 0.4, 0.2, 0.2])
```

Let's practice!

INTRODUCTION TO PORTFOLIO RISK MANAGEMENT IN PYTHON

Correlation and co-variance

INTRODUCTION TO PORTFOLIO RISK MANAGEMENT IN PYTHON



Dakota Wixom

Quantitative Analyst | QuantCourse.com

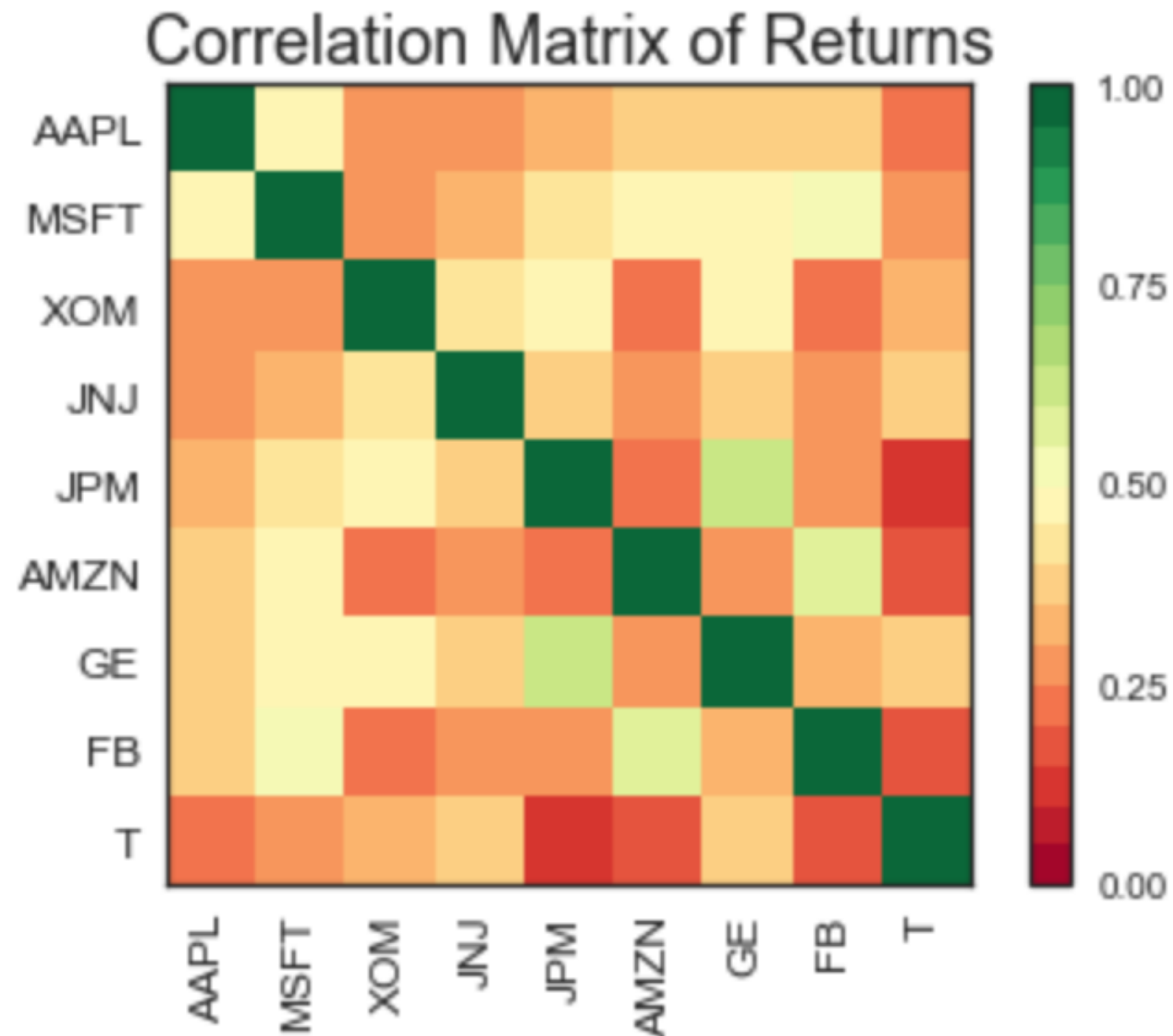
Pearson correlation

Examples of different correlations between two random variables:



Pearson correlation

A heatmap of a correlation matrix:



Correlation matrix in Python

Assuming `StockReturns` is a `pandas` `DataFrame` of stock returns, you can calculate the correlation matrix as follows:

```
correlation_matrix = StockReturns.corr()  
print(correlation_matrix)
```

	AAPL	MSFT	XOM
AAPL	1.000000	0.494160	0.272386
MSFT	0.494160	1.000000	0.289405
XOM	0.272386	0.289405	1.000000

Portfolio standard deviation

Portfolio standard deviation for a two asset portfolio:

$$\sigma_p = \sqrt{w_1^2 \sigma_1^2 + w_2^2 \sigma_2^2 + 2w_1 w_2 \rho_{1,2} \sigma_1 \sigma_2}$$

- σ_p : Portfolio standard deviation
- w : Asset weight
- σ : Asset volatility
- $\rho_{1,2}$: Correlation between assets 1 and 2

The Co-variance matrix

To calculate the co-variance matrix (Σ) of returns X :

$$\Sigma = \begin{bmatrix} E[(X_1 - \mu_1)(X_1 - \mu_1)] & E[(X_1 - \mu_1)(X_2 - \mu_2)] & \cdots & E[(X_1 - \mu_1)(X_n - \mu_n)] \\ E[(X_2 - \mu_2)(X_1 - \mu_1)] & E[(X_2 - \mu_2)(X_2 - \mu_2)] & \cdots & E[(X_2 - \mu_2)(X_n - \mu_n)] \\ \vdots & \vdots & \ddots & \vdots \\ E[(X_n - \mu_n)(X_1 - \mu_1)] & E[(X_n - \mu_n)(X_2 - \mu_2)] & \cdots & E[(X_n - \mu_n)(X_n - \mu_n)] \end{bmatrix}$$

The Co-variance matrix in Python

Assuming `StockReturns` is a `pandas` `DataFrame` of stock returns, you can calculate the covariance matrix as follows:

```
cov_mat = StockReturns.cov()  
cov_mat
```

	AAPL	MSFT	XOM	JNJ	JPM	AMZN	GE	FB	T
AAPL	0.000216	0.000104	0.000048	0.000033	0.000080	0.000097	0.000059	0.000093	0.000031
MSFT	0.000104	0.000204	0.000050	0.000040	0.000098	0.000133	0.000076	0.000132	0.000034
XOM	0.000048	0.000050	0.000145	0.000042	0.000090	0.000055	0.000059	0.000049	0.000037
JNJ	0.000033	0.000040	0.000042	0.000072	0.000047	0.000043	0.000037	0.000043	0.000028
JPM	0.000080	0.000098	0.000090	0.000047	0.000241	0.000073	0.000106	0.000073	0.000020
AMZN	0.000097	0.000133	0.000055	0.000043	0.000073	0.000350	0.000058	0.000197	0.000034
GE	0.000059	0.000076	0.000059	0.000037	0.000106	0.000058	0.000117	0.000065	0.000037
FB	0.000093	0.000132	0.000049	0.000043	0.000073	0.000197	0.000065	0.000319	0.000025
T	0.000031	0.000034	0.000037	0.000028	0.000020	0.000034	0.000037	0.000025	0.000083

Annualizing the covariance matrix

To annualize the covariance matrix:

```
cov_mat_annual = cov_mat * 252
```

Portfolio standard deviation using covariance

The formula for portfolio volatility is:

$$\sigma_{Portfolio} = \sqrt{w_T \cdot \Sigma \cdot w}$$

- $\sigma_{Portfolio}$: Portfolio volatility
- Σ : Covariance matrix of returns
- w : Portfolio weights (w_T is transposed portfolio weights)
- $\cdot$: The dot-multiplication operator

Matrix transpose

Examples of matrix transpose operations:

$$\begin{bmatrix} 1 & 2 \end{bmatrix}^T = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}^T = \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}^T = \begin{bmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{bmatrix}$$

Dot product

The **dot product** operation of two vectors **a** and **b**:

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^n a_i b_i = a_1 b_1 + a_2 b_2 + \cdots + a_n b_n$$

Portfolio standard deviation using Python

To calculate portfolio volatility assume a weights array and a covariance matrix:

```
import numpy as np
port_vol = np.sqrt(np.dot(weights.T, np.dot(cov_mat, weights)))
port_vol
```

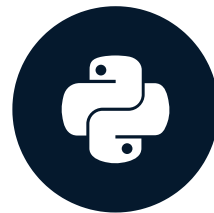
```
0.035
```


Let's practice!

INTRODUCTION TO PORTFOLIO RISK MANAGEMENT IN PYTHON

Markowitz portfolios

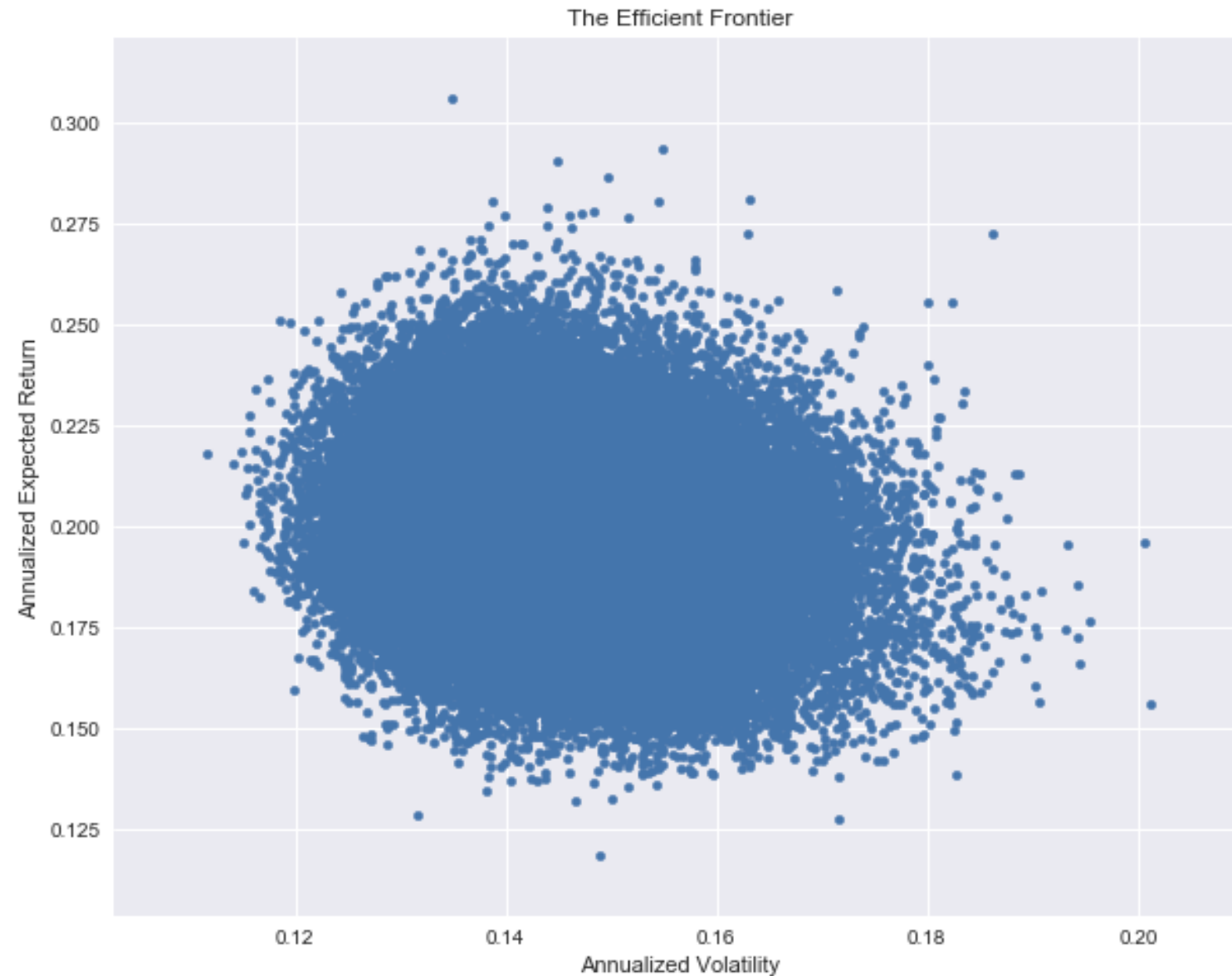
INTRODUCTION TO PORTFOLIO RISK MANAGEMENT IN PYTHON



Dakota Wixom

Quantitative Analyst | QuantCourse.com

100,000 randomly generated portfolios



Sharpe ratio

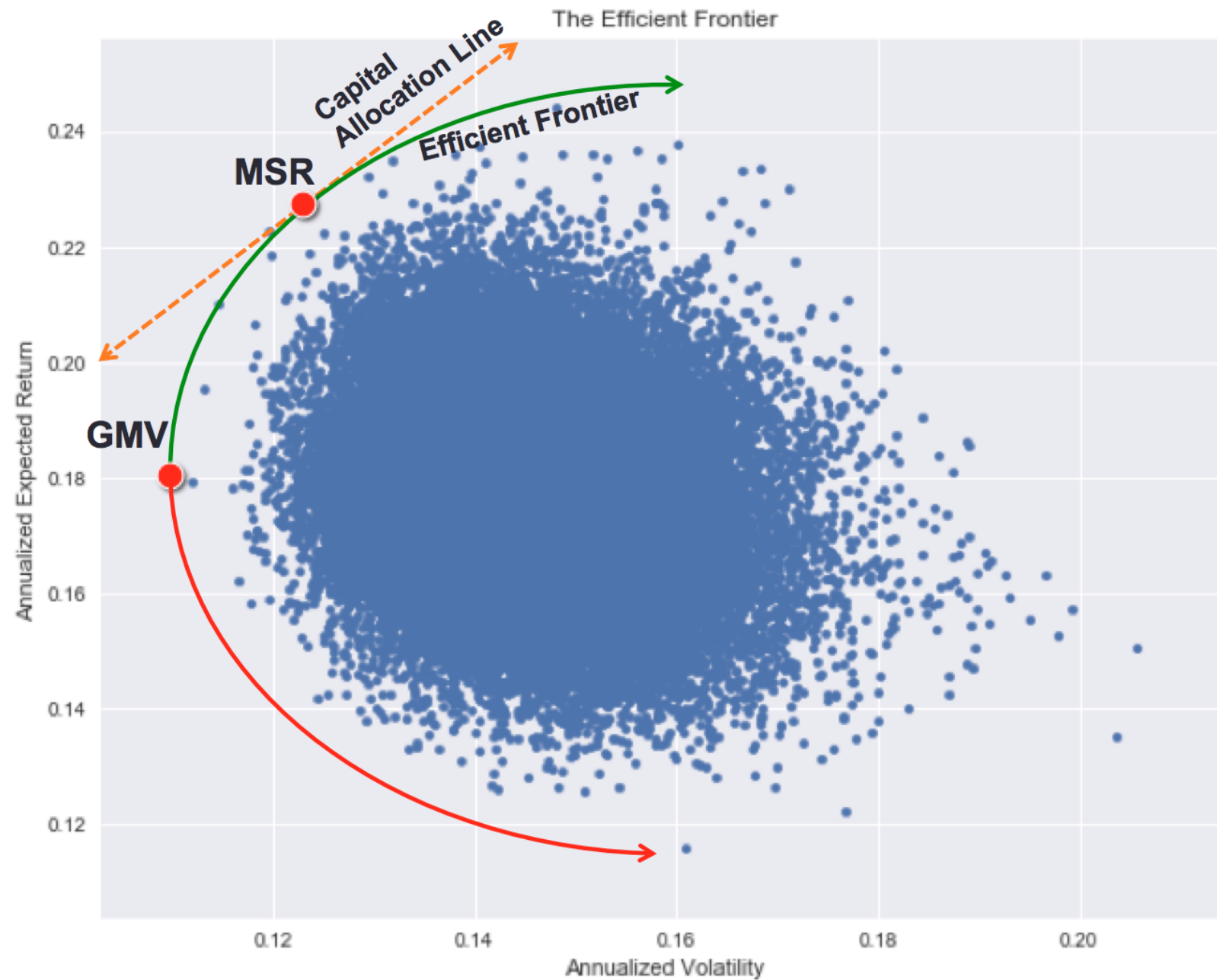
The Sharpe ratio is a measure of **risk-adjusted return**.

To calculate the 1966 version of the Sharpe ratio:

$$S = \frac{R_a - r_f}{\sigma_a}$$

- S : Sharpe Ratio
- R_a : Asset return
- r_f : Risk-free rate of return
- σ_a : Asset volatility

The efficient frontier

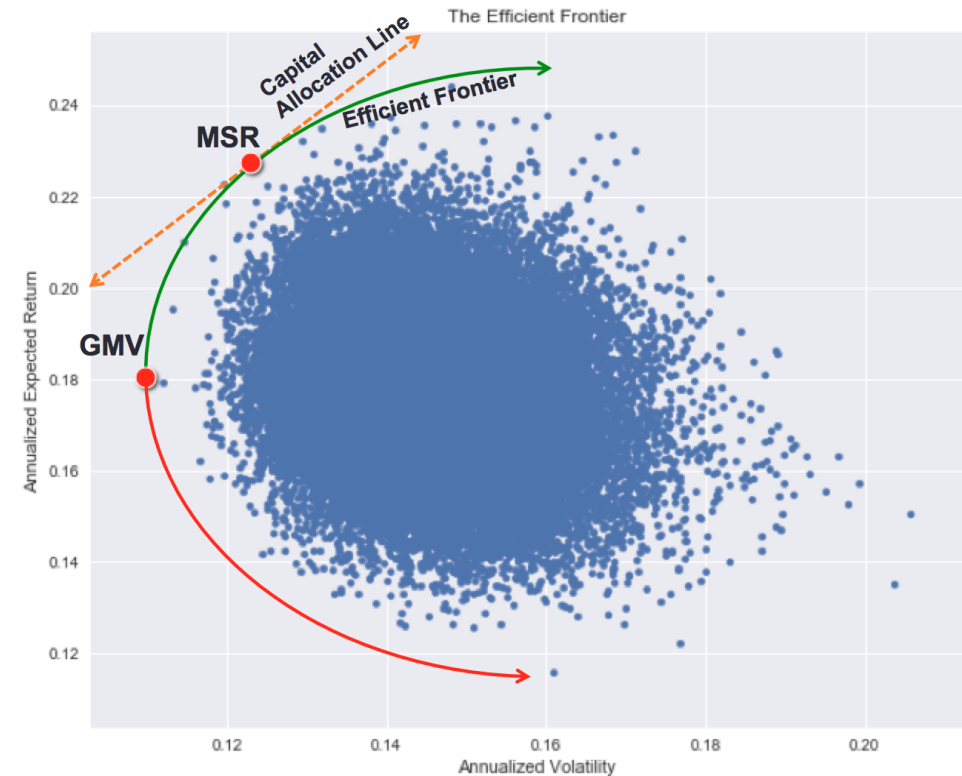


The Markowitz portfolios

Any point on the efficient frontier is an optimum portfolio.

These two common points are called **Markowitz Portfolios**:

- **MSR**: Max Sharpe Ratio portfolio
- **GMV**: Global Minimum Volatility portfolio



Choosing a portfolio

How do you choose the best portfolio?

- Try to pick a portfolio on the bounding edge of the efficient frontier
- Higher return is available if you can stomach higher risk

Selecting the MSR in Python

Assuming a DataFrame `df` of random portfolios with `Volatility` and `Returns` columns:

```
numstocks = 5
risk_free = 0
df["Sharpe"] = (df["Returns"] - risk_free) / df["Volatility"]
MSR = df.sort_values(by=['Sharpe'], ascending=False)
MSR_weights = MSR.iloc[0, 0:numstocks]
np.array(MSR_weights)
```

```
array([0.15, 0.35, 0.10, 0.15, 0.25])
```


Past performance is not a guarantee of future returns

Even though a Max Sharpe Ratio portfolio might sound nice, in practice, returns are extremely difficult to predict.

Selecting the GMV in Python

Assuming a DataFrame `df` of random portfolios with `Volatility` and `Returns` columns:

```
numstocks = 5
GMV = df.sort_values(by=['Volatility'], ascending=True)
GMV_weights = GMV.iloc[0, 0:numstocks]
np.array(GMV_weights)
```

```
array([0.25, 0.15, 0.35, 0.15, 0.10])
```

Let's practice!

INTRODUCTION TO PORTFOLIO RISK MANAGEMENT IN PYTHON